Proj 2. Mocking GPIO and BSRR

Jianhua Liu

Spring 2026

Contents

2.1 Introduction . . . 1
2.2 GPIO and BSRR mocking . . . 2
2.2.1 Principle of BSRR . . . 2
2.2.2 Mocking a GPIO port and its pointer . . . 3
2.2.3 Utility functions to access GPIO registers . . . 3
2.3 Getting started with the project (5 pts) . . . 4
2.4 Test cases for the project . . . 4
2.5 Programming task (PT) of the project (85 pts) . . . 6
2.5.1 PT 1. Mocking the updating of the IDR (15 pts) . . . 6
2.5.2 PT 2. Mocking the updating of ODR using BSRR (45 pts) . . . 7
2.5.3 PT 3. Testing a revised GPIO toggle function (25 pts) . . . 7
2.6 Submission of work (10 pts) . . . 8

2.1 Introduction

In Lctr 5, we have discussed how to use unit test to test functions that are mainly about calculations. In Lctrs 6 and 7, we have discussed how to configure GPIO functionalities and operate GPIO using HAL functions or direct GPIO register access. You may have a question: can we do unit test on functions that involve GPIO operations?

The answer is YES. The technique is to use mocking—a technique that **intercepts the access of real GPIO registers** and reroutes it to access **locally-defined mocked (faked) GPIO registers** for which we have FULL control.

Here we will practice how to mock the operation of GPIO. As we will see, many HAL functions for GPIO operations use the bit set and reset register (BSRR), which is designed in such a complex way that we can set or reset (clear) a bit in the output data register (ODR) by a simple bit set operation in BSRR. This means that **the bit values in BSRR can be automatically propagated to ODR**. Note that the same approach can be used to mock the bit reset register (BRR). Yet, BRR is not implemented in old processors, such as STM32F412xx. So, for our project to be able to run on all processors, we will only focus on the mocking of the operation of GPIO via BSRR.

Note that *mocking* is different than *simulation with Renode*, as listed below:

- Simulation with Renode uses exactly the same executables that are *cross-compiled* to run on real MPBs.
- While mocking can certainly work as an executable aimed to run on an MPB, it does not have to be cross-compiled for an MPB. It can be *natively compiled* on PC to run on PC. We just need to *mock* the functionality of some functions, including hardware-access functions.

We can certainly perform this project using native build. Yet, if we go this route, we have to set up the toolchain, which is doable for CubeIDE but time consuming. For convenience, we just use the existing toolchain for cross-compiling. So we will build for F412dsc to run on Renode or for G431n32 to run on the real MPB with full capability of debugging.

Note that the needed source files can be found in the enclosed `\de4u_src_test.zip` file.

2.2 GPIO and BSRR mocking

2.2.1 Principle of BSRR

We have discussed that the operation of GPIO is fully controlled by GPIO registers and that BSRR is one special GPIO register that can be used set or reset (clear) a bit of the ODR by writing a 1 to the corresponding bit of BSRR. Let's see how this is done in detail.

The bit map of BSRR is shown in Figure 1. The contents of this register is divided into two sections—the BS (bit set) section used for setting an ODR bit to 1, and the BR (bit reset or clear) section for clearing a bit of ODR to 0. Note that the content of this register is only writable.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Figure 1: Bit map of BSRR.

The reason for using this register is to save the time and code size for operation. To see this, let's take a look at the following two statements used to set PA10 and clear PB8 using standard ODR operations:

```
GPIOA->ODR |= mask_Pin10;  // mask_Pin10 = (1U << 10);
GPIOB->ODR &= ~mask_Pin8;  // mask_Pin8 = (1U << 8);
```

In reality, each of the above statements uses **three** steps, each with an ASM instruction, in the processor:

- Load (fetch) the value of ODR to a general-purpose register inside the processor core.
- Modify the value saved in the general-purpose register using the mask.
- Store (write) the modified value from the general-purpose register to the ODR.

Using the BSRR, we can use the following statements to perform the same work:

```
GPIOA->BSRR = mask_Pin10;  // mask_Pin10 = (1U << 10);
GPIOB->BSRR = mask_Pin8R;  // mask_Pin8R = (1U << (8+16));
```

Each of the above statements uses **two** steps, each with an ASM instruction, in the processor:

- Load the mask to a general-purpose register.
- Store the value from the general-purpose register to the BSRR.

Note that to reset a bit, we need to use a mask that is shifted to the left by 16 as the register bits for GPIO pin reset are on the upper 16 bits.

Note also that the same technique is widely use in many other MCU peripheral registers, such as the interrupt set enable register.

2.2.2 Mocking a GPIO port and its pointer

Regardless of native or cross compilation, to mock a GPIO port, we need to define (allocate memory space for) a variable representing the GPIO bank of registers. We also need to define its corresponding pointer. Here, we define the mocked bank of registers for GPIO and the pointer in a file named `mp_stm32xx_mocked_gpiox.c`, as shown below.

```
#include "mp_supported_mcu.h"

GPIO_TypeDef mp_mocked_gpiox;

GPIO_TypeDef *GPIOx = &mp_mocked_gpiox;
```

Note that the above definition is the key for GPIO mocking as we can use the newly-defined struct `mp_mocked_gpiox` to mimic the GPIO registers, and we can use `GPIOx` in the same as `GPIOA`, `GPIOB`, which we have used before. To use `GPIOx` in a source file, we need to include header file `mp_stm32xx_mocked_gpiox.h`.

The advantage of mocking is that we can use exactly the same functions as we have used before for GPIO access, and the **data will be written to `mp_mocked_gpiox`**, which we have full access. This way, we can access the data written to the registers to evaluate the GPIO operation functions.

2.2.3 Utility functions to access GPIO registers

When we use real GPIO hardware, the input data register (IDR) is read-only as its data is automatically changed by the GPIO inputs. When we use mocked GPIO registers, we need to have a way to change the value of each IDR bit to mock its automatic update according to the change of the digital input signal connected the corresponding pin. This is done using a utility function, called `mp_mock_write_idr()`.

Although ODR is both readable and writeable, we normally only use the write operation. Here, we provide a utility function, called `mp_mock_read_odr()`, to read the ODR so that we can see if the writing to the ODR is correct or not.

The prototype of these two functions are provided in the following header file, named `mp_mock_utils_4_gpio_regs.h`.

```
#pragma once

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

#include <stdint.h>
#include "mp_supported_mcu.h"
```

```

uint32_t mp_mock_read_odr(GPIO_TypeDef *GPIOx);

void mp_mock_write_idr(GPIO_TypeDef *GPIOx, uint32_t idr);

#ifdef __cplusplus
}
#endif /* __cplusplus */
```

The corresponding C implementations are given below, contained in the `mp_mock_utils_4_gpio_regs.c` file:

```
#include "mp_supported_mcu.h"

uint32_t mp_mock_read_odr(GPIO_TypeDef *GPIOx) {
    return GPIOx->ODR;
}

__attribute__((weak))
void mp_mock_write_idr(GPIO_TypeDef *GPIOx, uint32_t idr) {
    // Add your code here.
}
```

Note that the second function is not completed. This will be a programming task as detailed later.

2.3 Getting started with the project (5 pts)

The same as Proj 1, you can use the provided `ca4b_cls_projs` base project. Here, you need to put the source files in the `proj2_src` and `proj2_test` folders, respectively. Then, you need to link the C source files to the `proj2_src_test` folder in CubeIDE.

Now, build `Proj2Unity`. This should build without any problem. If you have issues, ask around.

Note that we will build two executables for each MPB, F412dsc and G431n32.

- App, using the `Proj2App` build configuration. Note that there is no meaningful code for the App.
- Unit test, using the `Proj2Unity` build configuration. This is the focus of the project.

Note that you need to create the corresponding Renode script file to run the `Proj2Unity` build with Renode.

2.4 Test cases for the project

Note that we will use both HAL functions and direct GPIO register access to perform GPIO operations. So, the test cases will cover both the above two approaches.

All the test functions are given in `test_de4u_mocking_gpio_n_bsrr.c`, which can be found in the enclosed `\de4u_src_test.zip` file.

```
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include "unity.h"
#include "mp_stm32xx_mocked_gpiox.h"
#include "mp_mock_utils_4_gpio_regs.h"
#include "mp_mock_odr_update_with_bsrr_brr.h"
#include "mp_gpio_toggle.h"
```

```c

//-----------------------------------------------
void test_read_idr_via_hal_func(void) {
    uint16_t act = 0, exp = 0xABCD;
    mp_mock_write_idr(GPIOx, exp);  // write to IDR as needed for testing
    uint16_t pin_state;
    for (int i = 0; i < 16; i++) {
        pin_state = HAL_GPIO_ReadPin(GPIOx, (1U<<i)); // read a pin
        act |= pin_state << i;
    }
    TEST_ASSERT_EQUAL_UINT16(exp, act);
}

//-----------------------------------------------
void test_read_idr_directly(void) {
    uint32_t act, exp = 0xABCD;
    mp_mock_write_idr(GPIOx, exp);  // write to IDR as needed for testing
    act = GPIOx->IDR;               // read IDR as normal
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

//-----------------------------------------------
void test_write_odr_via_hal_func(void) {
    uint32_t act, exp = 0xABCD;
    GPIOx->ODR = 0;                 // Clear ODR
    HAL_GPIO_WritePin(GPIOx, exp, GPIO_PIN_SET);  // Write to ODR as normal
    // The WritePin function only writes to BSRR; we need to update ODR
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);  // read from ODR as needed for testing
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

//-----------------------------------------------
void test_write_odr_directly(void) {
    uint32_t act, exp = 0xABCD;
    GPIOx->ODR = exp;               // Write to ODR as normal
    act = mp_mock_read_odr(GPIOx);  // read from ODR as needed for testing
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

//-----------------------------------------------
void test_write_bsrr_to_set_odr(void) {
    uint32_t exp = 0xABCD;
    uint32_t act;

    GPIOx->ODR = exp;
    // Wirte to lower halfword to set ODR
    GPIOx->BSRR = 0x1234;
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);
    // Expected value of ODR should be exp | 0x1234 now.
    TEST_ASSERT_EQUAL_UINT32(exp|0x1234, act);

    GPIOx->ODR = exp;
    // Now, check if the BSRR is cleared to 0 or not.
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

```

```c
//-----------------------------------------------
void test_write_bsrr_to_clear_odr(void) {
    uint32_t exp = 0xABCD;
    uint32_t act;

    GPIOx->ODR = exp;
    // Wirte to upper halfword to clear ODR
    GPIOx->BSRR = 0x1234 << 16;
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);
    // Expected value of ODR should be exp & ~0x1234 now.
    TEST_ASSERT_EQUAL_UINT32(exp & (~0x1234), act);

    GPIOx->ODR = exp;
    // Now, check if the BSRR is cleared to 0 or not.
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

//-----------------------------------------------
void test_hal_gpio_toggle_pin_via_hal_func(void) {
    // Here we test the toggle of Pins 4 to 7.
    uint32_t act, exp = 0xABCD, odr = 0xAB3D;
    GPIOx->ODR = odr;                        // Write to ODR as normal
    HAL_GPIO_TogglePin(GPIOx, (15U<<4));     // toggle Pins 4 to 7.
    mp_mock_odr_update_with_bsrr(GPIOx);
    act = mp_mock_read_odr(GPIOx);      // read from ODR as needed for testing
    TEST_ASSERT_EQUAL_UINT32(exp, act);
}

__attribute__((weak))
void test_mp_gpio_toggle_pins_func(void) {
}


void mp_unity(void) {
    UNITY_BEGIN();

    RUN_TEST(test_read_idr_via_hal_func);
    RUN_TEST(test_read_idr_directly);
    RUN_TEST(test_write_odr_via_hal_func);
    RUN_TEST(test_write_odr_directly);
    RUN_TEST(test_write_bsrr_to_set_odr);
    RUN_TEST(test_write_bsrr_to_clear_odr);
    RUN_TEST(test_hal_gpio_toggle_pin_via_hal_func);
    RUN_TEST(test_mp_gpio_toggle_pins_func);

    UNITY_END();

    while (1);
}
```

2.5 Programming task (PT) of the project (85 pts)

2.5.1 PT 1. Mocking the updating of the IDR (15 pts)

We have discussed before that the IDR is updated by the GPIO input automatically. For the mocking, we need to program a `mp_mock_write_idr` function to perform the updating.

Program this function so that you can pass **Tests 1 and 2**. No instruction is provided for this task. You need to read the two test functions `test_read_idr_via_hal_func` and `test_read_idr_directly` carefully to understand what is really needed. If you have questions, you can ask.

2.5.2 PT 2. Mocking the updating of ODR using BSRR (45 pts)

We have discussed before that the ODR can be updated by BSRR automatically. For the mocking, we need to program a `mp_mock_odr_update_with_bsrr` function to perform the updating.

We need to use this function after running the `HAL_GPIO_WritePin` function as the implementation of latter for F412dsc uses BSRR, as shown below (simplified version):

```
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState
    PinState) {
    if (PinState == GPIO_PIN_SET) {
        GPIOx->BSRR = GPIO_Pin;
    } else {
        GPIOx->BSRR = (uint32_t)GPIO_Pin << 16U;
    }
}
```

Note that the implementation for G431n32 is a bit different, but the difference does not matter.

Here are some hints for programming the `mp_mock_odr_update_with_bsrr` function:

- In this function, whenever the value of the BSRR is not zero, we need to update the corresponding value of ODR.
- After the updating, we need to **clear every bit** of BSRR so that they will not affect the next update operation.

Similarly, no further instruction is provided for this task. You need to fully understand the discussion in the Principle of BSRR section. With that, you should be able to figure out the strategy to program this function. If you have questions, you can ask.

When this function is written correctly, you will be able to pass Tests 3, 5, 6, and 7. Test 4 will automatically pass.

2.5.3 PT 3. Testing a revised GPIO toggle function (25 pts)

Here, we revise the provided HAL GPIO toggle function and test it.

First, let's take a look at the steps needed for a normal ODR operation. This is done in three steps:

1. Load ODR to a general purpose register.
2. Operate on this register as needed.
3. Store the content of this register to ODR.

Now, let's take a look at the code in the HAL GPIO toggle pin function:

```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
  uint32_t odr;

  /* Check the parameters */
```

```
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  /* get current Output Data Register value */
  odr = GPIOx->ODR;

  /* Set selected pins that were low, and reset ones that were high */
  GPIOx->BSRR = ((odr & GPIO_Pin) << GPIO_NUMBER) | (~odr & GPIO_Pin);
}
```

There are two issues with the above code:

1. It reads the value of ODR first and then use BSRR! This defeats the purpose of using BSRR!
2. The way of assigning values to BSRR is very clever but difficult to understand to most programmers. Code like this can be a good place to hide bugs.

Here is the revised toggle function:

```
void mp_GPIO_TogglePinS(GPIO_TypeDef* GPIOx, uint16_t GPIO_PinS) {
    /* Check the parameters */
    assert_param(IS_GPIO_PIN(GPIO_PinS));

    /* Toggle the pins using the bit mask GPIO_Pins */
    GPIOx->ODR ^= GPIO_PinS;
}
```

Note that we have changed the name of the function as well, as this function works on multiple bits without any problem.

Here are the takeaways:

- A working function may not be implemented in the best way. If possible, we always try to improve the function.
- To make sure our revision still executes correctly, we need to test it using unit test or mocking.

Modify Test 7 to create Test 8 titled `void test_mp_gpio_toggle_pin_via_mp_func` to test the above `mp_GPIO_TogglePinS` function. Remove all unnecessary code in the test file. Specifically, you need to consider if you still need to use `mp_mock_odr_update_with_bsrr(GPIOx)`.

2.6 Submission of work (10 pts)

Your report should include the following:

- The code you wrote for the functions to pass the tests.
- The screenshot that shows that you have passed all the tests, including the newly created one.

Here are what you need to submit:

- Your brief report named as `de4u-report-lastname-firstname.pdf`.
- Your project zipped as `de4u-lastname-firstname.zip`. Before zipping your project, clean out all builds you have created in CubeIDE. This is especially important as there can be multiple built executable at this point, some are from previous project assignments.